OOP SOLUTIONS

Mobile Shop

Operations Runbook (internal)

Our own runbook: GitHub, branches, releases, the server, environment variables, every command we run, backups and what to do when something breaks.

OOP Solutions · mobileshop.oopsolutions.com · 2026-11-03

Contents

1. The pieces and where they live
2. Access you need
3. Environment variables
4. Daily development commands (Windows)
5. Git and GitHub
6. Releasing to production
7. The server
8. Backups and restore
9. Android app and the Windows PC version
10. Demo data, landing screenshots and the manuals
11. Monitoring and health checks
12. When something breaks
13. Still pending on the owner's side

This is our own manual for running Mobile Shop day to day: where every piece lives, which access you need, every environment variable, every command we run, how a release goes live, how backups work and what to do when something breaks. It is written for the owner, Abdur Rehman Afzal, and for any team member who takes over operations.

> **Important:** Real passwords, tokens, API keys and the server's IP address are never written in this document or anywhere in the repository. They live only in three places: the server's `/opt/mobileshop/.env` (root only), GitHub → repository → Settings → Secrets and variables → Actions, and the owner's password manager. In this runbook they appear as placeholders such as `<VPS_IP>`, `<SSH_USER>` or `<WHATSAPP_TOKEN>`.

How to read the commands: every command is on its own line in `inline code`. "PowerShell" means the Windows PC in the office; "Git Bash" means the Bash shell that comes with Git for Windows; "server" means an SSH session on the VPS as root, inside `/opt/mobileshop` unless stated otherwise.

1. The pieces and where they live

The repository

- **GitHub repository:** `https://github.com/AbdurRehmanAfzal/mobile-shop-software` (remote `origin`, the only remote).
- **Local copy on the office PC:** `C:\Users\HP\Documents\projects\mobile shop\mobile-shop-software`.
- **Layout:** `backend/` (FastAPI, Python 3.12, SQLAlchemy 2, Alembic, pytest), `frontend/` (React, Vite, TypeScript, Tailwind, PWA, Capacitor Android project in `frontend/android`), `deploy/` (production stack), `local/` (the one-PC Windows program), `.github/workflows/` (CI and releases), `docs/` (living documentation), `legacy/` (the old v1, reference only — never extended).
- **Project context for Claude Code:** `CLAUDE.md` at the root. Its hard rules apply to everyone, not only to Claude.

Branches

Branch	Purpose	What a push does
`develop`	All daily work. Every commit lands here first.	Runs the CI workflow (checks only, nothing ships).
`main`	Releases only. Changes arrive by a pull request from `develop`.	Runs the Deploy workflow: checks, Android APK, Docker images, deploy to the VPS. Also runs the Windows workflow when `local/`, `backend/` or `frontend/` changed.

`origin/HEAD` points to `main`. There are no other long-lived branches. Feature branches are allowed but are not part of the normal flow; we commit straight to `develop`.

GitHub Actions workflows

File	Name in the Actions tab	Triggered by	What it does
`.github/workflows/ci.yml`	CI	push to `develop`, any pull request, or called by Deploy	Jobs `backend` (ruff, pytest on SQLite, `alembic upgrade head` / `downgrade base` / `upgrade head` on PostgreSQL 16), `frontend` (typecheck, vitest, production build), `stack` (builds the production images, runs db + backend + web together on the runner, runs `deploy/ci/smoke.sh`).
`.github/workflows/deploy.yml`	Deploy	push to `main`, or by hand (Run workflow)	Jobs `checks` (the whole CI, shown as `checks / backend`, `checks / frontend`, `checks / stack`), `android` (calls android.yml), `images` (pushes backend and web images to ghcr.io), `deploy` (SSH to the VPS, runs `deploy.sh <sha>`, then checks `APP_URL/health`).
`.github/workflows/android.yml`	Android	called by Deploy, or by hand	Builds the Capacitor app against `vars.APP_URL`, signs it with the release key if the secrets exist (else a debug "test" APK), uploads it as an artifact kept 90 days.
`.github/workflows/windows.yml`	Windows	by hand (with a version number), or a push to `main` that touches `local/`, `backend/`, `frontend/` or the workflow	Builds the one-PC program with PyInstaller and Inno Setup, smoke-tests `/health`, uploads `MobileShop-Setup-<version>.exe` as an artifact.

Concurrency: CI runs cancel an older run on the same branch; Deploy runs never cancel each other (group `deploy`), they queue.

The production server (VPS)

- **Provider:** Hostinger VPS, shared with the company website (oopsolutions.com), n8n and the voice agent. Address: `<VPS_IP>` (kept in GitHub secret `VPS_HOST` and in the password manager).
- **Users:** `root` (you, for maintenance) and `deploy` (used only by GitHub Actions; its sudo rule allows exactly one command, `/opt/mobileshop/deploy.sh`).
- **App folder:** `/opt/mobileshop` holds `docker-compose.yml`, `deploy.sh`, `backup.sh`, `.env` (mode 600, root only) and `.current-tag` (the commit sha that is live now).
- **Backups folder:** `/opt/mobileshop-backups`. Backup log: `/var/log/mobileshop-backup.log`.
- **Sudoers rule:** `/etc/sudoers.d/mobileshop-deploy`.

Domain, HTTPS and routing

- **Production domain:** `mobileshop.oopsolutions.com` (DNS A record in the Hostinger panel → `<VPS_IP>`).
- **Public pages:** `/` is the app (login), `/welcome` is the landing page, `/app.apk` is the Android app, `/robots.txt`, `/sitemap.xml` and `/llms.txt` are for search and answer engines, `/health` answers `{"status":"ok","env":"prod"}` when the backend is up.
- **HTTPS:** the VPS already runs Traefik (container `n8n-traefik-1`, Docker network `n8n_default`). Traefik owns ports 80 and 443, gets the Let's Encrypt certificate (resolver `mytlschallenge`) and redirects plain HTTP to HTTPS. Our stack opens no ports of its own.
- **Caddy:** inside our `web` container. It serves the built React app on plain port 80, passes `/api/*` and `/health` to the backend, serves `/app.apk` as an Android package download, sets the security headers (HSTS, nosniff, frame and referrer policy, camera allowed only for this site) and limits request bodies to 10 MB. Config: `deploy/Caddyfile`.

Docker services (compose project `mobileshop`)

Service	Image	Networks	Role
`db`	`postgres:16-alpine`	`internal` only	The database. Data in volume `mobileshop_pgdata`. Health check `pg_isready`.

Service	Image	Networks	Role
backend	ghcr.io/abdurrehmanafzal/mobile-shop-backend:<sha>	internal (alias mobileshop-backend)	FastAPI with uvicorn (WEB_WORKERS, default 2). On every start it runs alembic upgrade head, then seeds the phone catalogue and expense categories (idempotent). Uploads in volume mobileshop_uploads at /data/uploads. Runs the in-process scheduler (night reports, reminders, queued messages).
web	ghcr.io/abdurrehmanafzal/mobile-shop-web:<sha>	internal and n8n_default	Caddy with the built React app and the APK. Carries the Traefik labels for the domain.

The images are built by the Deploy workflow and pushed to GitHub's container registry (ghcr.io) with two tags each: the commit sha and `latest`. The server always runs a sha tag; `deploy.sh` writes it to `.current-tag`.

Database and uploads

- **Production:** PostgreSQL 16, database `mobileshop`, user `mobileshop` (both from `.env`), reachable only from inside the stack.
- **Local development:** SQLite file `backend/mobile_shop_v2.db` (git-ignored).
- **Migrations:** Alembic, in `backend/alembic/versions` (48 files today). The current head is `a5c7e9b1d3f6` (repair dispatches). Migrations are additive and run automatically when the backend container starts.
- **Uploads:** CNIC photos, signatures, fee screenshots, logos. Production: volume `mobileshop_uploads`. Local: `backend/uploads/`. They are sensitive (identity documents) — treat every copy accordingly.

Backups

- A cron job at 02:30 server time runs `/opt/mobileshop/backup.sh`: a `pg_dump` in custom format (`db-YYYYMMDD-HHMM.dump`) and a tar of the uploads volume (`uploads-YYYYMMDD-HHMM.tgz`) into `/opt/mobileshop-backups`, kept 14 days.
- When `RCLONE_REMOTE` is set in `.env`, the same job copies the new files to an encrypted Google Drive remote and keeps 30 days there. As of today this is **not set up yet** (see chapter 13).

The Android app

- Built by every release (the `android` job), signed with the release key when the GitHub secrets exist, and baked into the web image, so the newest app is always at `https://mobileshop.oopsolutions.com/app.apk`.
- App id `pk.mobileshop.app`, Capacitor 6. Version code = minutes since 1970 (always higher than the previous build); version name = date plus short commit sha.
- Each run also keeps the APK as an artifact named `mobile-shop-android-<date>-<sha>` for 90 days.

The Windows one-PC program

For shops without reliable internet: the same backend with SQLite on the shop's PC, serving the same web app at `http://127.0.0.1:8787`. Built by the Windows workflow or `local\build.ps1`. Data lives on the shop's PC in `%LOCALAPPDATA%\Mobile Shop\MobileShop\`. Details in `docs/15-LOCAL-WINDOWS.md`.

The company website

`oopsolutions.com` is a separate repository (`oop-solutions`, built with Astro). A push to its `master` branch deploys it to the same VPS. It shares Traefik and the `deploy` user with Mobile Shop; its own `deploy/setup-vps.sh` created that user, so it must exist before Mobile Shop's setup can run.

2. Access you need

Keep one entry per row in the password manager. Never share these in WhatsApp, email or a commit.

Access	What it is for	Who has it	Where the real value is kept
GitHub account `AbdurRehmanAfzal`	Repository, Actions, pull requests, secrets, packages (ghcr.io)	Owner	Password manager (plus 2FA device)
GitHub CLI login on the office PC	`gh pr`, `gh run` from PowerShell	Owner's PC	`gh auth login` stores it in the Windows credential store
GitHub personal access token with `read:packages`	Only if the ghcr.io images are private: `docker login ghcr.io` on the VPS	Owner	Password manager; Docker keeps it in `/root/.docker/config.json` on the VPS
VPS root SSH (`<SSH_USER>@<VPS_IP>`)	Server maintenance, `.env`, logs, backups, restores	Owner	Password manager (SSH key or root password)
VPS `deploy` user SSH key	GitHub Actions deploys	GitHub only	GitHub secret `VPS_SSH_KEY`; public half in `/home/deploy/.ssh/authorized_keys`
Hostinger panel	VPS console, reboot, snapshots, DNS for oopsolutions.com	Owner	Password manager
Google Search Console	Indexing of `/welcome`, sitemap submission	Owner	Owner's Google account
Meta Business / WhatsApp Cloud API	Automatic WhatsApp (reminders, night report, OTP)	Owner	`WHATSAPP_TOKEN`, `WHATSAPP_PHONE_ID` in server `.env`; master copy in password manager
Anthropic API (Claude)	The in-app assistant and photo reading	Owner	`ANTHROPIC_API_KEY` in server `.env`; console login in password manager

Access	What it is for	Who has it	Where the real value is kept
SMTP account (e.g. Gmail app password)	Outgoing email	Owner	`SMTP_*` in server `.env`; password manager
Google Drive + rclone crypt passwords	Off-server backup copy	Owner	rclone config on the VPS (`/root/.config/rclone/rclone.conf`); the two crypt passwords in the password manager (without them the backups cannot be opened)
Android release keystore (`mobileshop-release.jks`) and its password	Signing every app update	Owner	GitHub secrets `ANDROID_KEYSTORE_BASE64`, `ANDROID_KEYSTORE_PASSWORD`, `ANDROID_KEY_ALIAS`; a copy of the file and the password in the password manager, off the server
Renewal / license private key	Signs renewal codes and license keys	Server only	`RENEWAL_PRIVATE_KEY` in server `.env`; optional sealed copy in the password manager
Super-admin login in the app	More → Super admin, Business console	Owner	The number is in `SUPERUSER_PHONES`; the password in the password manager

> **Important:** When a team member leaves, rotate what they could see: the SSH key, the GitHub secrets they touched, and any API key that was in a file they opened.

3. Environment variables

There are four places variables come from:

- **Server stack:** `/opt/mobileshop/.env`, read by `docker compose --env-file .env`. The template is `deploy/.env.example`. Only keys that `deploy/docker-compose.yml` passes into a container have any effect; a key you add to `.env` that the compose file does not pass is silently ignored.
- **Backend locally:** `backend/.env` (git-ignored; template `backend/.env.example`). Every setting in `backend/app/core/config.py` can be set here, names are case-insensitive.
- **Frontend build:** `VITE_API_URL` (template `frontend/.env.example`), set at build time.
- **GitHub Actions:** repository secrets and variables.

Server stack (`/opt/mobileshop/.env`)

Name	Used by	Required?	Example (not a real value)	What it does	Where the real value is kept
`APP_DOMAIN`	web (Traefik labels), backend (CORS, passkeys)	Yes	`mobileshop.oopsolutions.com`	The public address. Builds `CORS_ORIGINS`, `WEBAUTHN_RP_ID` and `WEBAUTHN_ORIGINS`.	Server `.env`
`POSTGRES_DB`	db, backend	No (default `mobileshop`)	`mobileshop`	Database name.	Server `.env`
`POSTGRES_USER`	db, backend, backup.sh	No (default `mobileshop`)	`mobileshop`	Database user.	Server `.env`
`POSTGRES_PASSWORD`	db, backend	Yes	`<POSTGRES_PASSWORD>` (`openssl rand -hex 24`)	Database password; `setup-vps.sh` generates it. Never change it after the first start without changing it inside PostgreSQL too.	Server `.env`; password manager

Name	Used by	Required?	Example (not a real value)	What it does	Where the real value is kept
`SECRET_KEY`	backend	Yes	`<SECRET_KEY>` (`openssl rand -hex 32`)	Signs login tokens. With `APP_ENV=prod` the backend refuses to start if it is a default or shorter than 32 characters. Changing it logs everyone out.	Server `.env` ; password manager
`WEB_WORKERS`	backend	No (default 2)	`2`	uvicorn worker processes.	Server `.env`
`PUBLIC_SIGNUP`	backend	No (default `false`)	`false`	When false, `/api/v1/auth/signup` returns 403; only the super admin opens shops. Keep it false on production.	Server `.env`
`SUPERUSER_PHONES`	backend	Yes	`03001234567`	Comma-separated login numbers that get More → Super admin and the Business console.	Server `.env`
`IMAGE_PREFIX`	compose	No	`ghcr.io/abdurrehmanafzal/mobile-shop`	Where the images come from.	Server `.env`
`IMAGE_TAG`	compose	Set per command	`3313730…`	Which image version runs. `deploy.sh` sets it; it is not stored in `.env` . Without it compose uses `latest` .	`.current-tag` on the server
`ADMIN_WHATSAPP`	backend	No	`03001234567`	Receives "fee proof submitted", "new shop request" and forgot-password requests. Empty = the first `SUPERUSER_PHONES` number.	Server `.env`

Name	Used by	Required?	Example (not a real value)	What it does	Where the real value is kept
`WHATSAPP_TOKEN`	backend	No	`<WHATSAPP_TOKEN>`	Meta permanent system-user token. With `WHATSAPP_PHONE_ID` set, messages go out by themselves; empty = sent by hand from More → Messages.	Server `.env`; password manager
`WHATSAPP_PHONE_ID`	backend	No	`<WHATSAPP_PHONE_ID>`	Meta "Phone number ID" of the business number.	Server `.env`
`WHATSAPP_TEMPLATES`	backend	No	`{"udhaar_reminder":{"name":"udhaar_reminder_ur","lang":"ur"}}`	JSON map from our message key to an approved Meta template. Keys without a template go as plain text.	Server `.env`
`SIGNUP_OTP`	backend	No (default `false`)	`false`	Public signup must prove the number by a WhatsApp code.	Server `.env`
`OTP_DEV_ECHO`	backend	No (default `false`)	`false`	Returns the OTP in the API response. **Never true on production.**	Server `.env`
`SMTP_HOST`	backend	No	`smtp.gmail.com`	Outgoing mail server. Empty = emails wait in Messages.	Server `.env`
`SMTP_PORT`	backend	No (default 587)	`587`	Mail server port.	Server `.env`
`SMTP_USER`	backend	No	`<SMTP_USER>`	Mail login.	Server `.env`
`SMTP_PASSWORD`	backend	No	`<SMTP_PASSWORD>`	Mail password (Gmail: an app password).	Server `.env`; password manager
`SMTP_FROM`	backend	No	`<SMTP_FROM>`	Sender address.	Server `.env`
`SMTP_STARTTLS`	backend	No (default `true`)	`true`	Use STARTTLS.	Server `.env`

Name	Used by	Required?	Example (not a real value)	What it does	Where the real value is kept
`ANTHROPIC_API_KEY`	backend	No	`<ANTHROPIC_API_KEY>`	Turns on the assistant and photo reading (CNIC, box label). Empty = "not switched on".	Server `.env` ; password manager
`AI_MODEL`	backend	No	`claude-opus-5-5`	Model for the chat assistant.	Server `.env`
`AI_VISION_MODEL`	backend	No	`claude-haiku-5-5`	Model for photo reading.	Server `.env`
`AI_DAILY_LIMIT`	backend	No (default 200)	`200`	AI calls per shop per day.	Server `.env`
`RENEWAL_PRIVATE_KEY`	backend	Yes, for keys and renewal codes	`<RENEWAL_PRIVATE_KEY>`	Signs renewal codes and license keys. Only ever on this server. Without it Business → Keys fails with "RENEWAL_PRIVATE_KEY is not set".	Server `.env` only
`RENEWAL_PUBLIC_KEY`	backend	Yes, with the private key	`<RENEWAL_PUBLIC_KEY>`	Verifies codes. Every one-PC install carries the same public key.	Server `.env` ; also baked into each one-PC install
`RCLONE_REMOTE`	backup.sh (not a container)	No	`gdrive-safe:`	Off-server copy of each night's backup.	Server `.env`
`BACKUP_DIR`	backup.sh	No (default `/opt/mobileshop-backups`)	`/opt/mobileshop-backups`	Where backups are written.	Environment of the cron line

Set by the compose file itself (do not put them in `.env`): `APP_ENV=prod` , `DATABASE_URL` (built from the Postgres keys), `CORS_ORIGINS` (the domain plus `https://localhost` and `capacitor://localhost` for the Android app), `UPLOAD_DIR=/data/uploads` , `WEBAUTHN_RP_ID` , `WEBAUTHN_ORIGINS` .

Backend settings not passed on the server

These exist in `backend/app/core/config.py` and work in `backend/.env` locally or in a one-PC install's `settings.env` , but `deploy/docker-compose.yml` does not pass them, so they keep their defaults on production unless the compose file is changed.

Name	Default	What it does
APP_ENV	dev	dev, test or prod. prod hides /docs and enforces a strong SECRET_KEY.
DATABASE_URL	sqlite:///./mobile_shop_v2.db	Database. PostgreSQL form: postgresql+psycopg://<user>:<password>@<host>:5432/mobileshop.
ACCESS_TOKEN_MINUTES	30	Login token lifetime.
REFRESH_TOKEN_DAYS	30	Refresh token lifetime.
CORS_ORIGINS	["http://localhost:5173","http://127.0.0.1:5173"]	Allowed browser origins (JSON list).
WEBAUTHN_RP_ID	localhost	Passkey (fingerprint login) domain.
WEBAUTHN_ORIGINS	http://localhost:5173,https://localhost:5174	Passkey origins.
UPLOAD_DIR	./uploads	Upload folder.
DEFAULT_LOCALE	ur	Language for new shops.
TIMEZONE	Asia/Karachi	Business day and scheduler time zone.
WHATSAPP_API_VERSION	v21.0	Meta Graph API version.
SCHEDULER_ENABLED	true	The in-process clock (night report, reminders, sending queued messages). Set false locally so your PC does not send messages.
WEB_DIR	empty	One-PC install: the backend also serves the built web app from this folder.
LOCAL_MODE	false	One-PC install: the shop is chain with no expiry.
SAAS_URL	empty	One-PC install: reports version and plan to the SaaS server's Business console when online.
APP_VERSION	2.0	Version reported by a one-PC install.

Local development (backend/.env on the office PC)

Name	Value locally	Why
APP_ENV	dev	Shows /docs, relaxed checks.
DATABASE_URL	sqlite:///./mobile_shop_v2.db	Local SQLite file.
SECRET_KEY	any long random string	Dev only.
PUBLIC_SIGNUP	true	Tests, e2e and the screenshot scripts sign up throwaway shops.
SUPERUSER_PHONES	your test number plus 03990000099	03990000099 is the e2e and admin-screenshot super user.
SCHEDULER_ENABLED	false	No real WhatsApp or email from the dev PC.
OTP_DEV_ECHO	true only if you are testing OTP	Code comes back in the response.
RENEWAL_PRIVATE_KEY / RENEWAL_PUBLIC_KEY	a dev pair	Never the production pair.

One-PC install (settings.env)

Name	What it does
MOBILESHOP_DATA	Data folder (default %LOCALAPPDATA%\Mobile Shop\MobileShop).
MOBILESHOP_PORT	Port (default 8787).
MOBILESHOP_HOST	127.0.0.1 by default; 0.0.0.0 opens it to phones on the shop Wi-Fi.
SAAS_URL	https://mobileshop.oopsolutions.com so the install reports to the Business console (pending decision, chapter 13).
plus any backend name above	Same names as deploy/.env.example (WhatsApp, SMTP, AI). A random SECRET_KEY is written on first run.

Frontend

Name	Where	What it does
VITE_API_URL	build time	Empty = same origin (/api), used by the web image, the dev server proxy (/api → http://localhost:8000) and the one-PC build. For the Android app it is the full site address (from the GitHub variable APP_URL).

GitHub Actions secrets and variables

Repository → Settings → Secrets and variables → Actions.

Name	Kind	Read by	What it holds	Where the real value is kept
VPS_HOST	Secret	deploy.yml deploy job	<VPS_IP>	GitHub; password manager
VPS_USER	Secret	deploy.yml deploy job	deploy	GitHub
VPS_SSH_KEY	Secret	deploy.yml deploy job	Private key of the deploy user	GitHub only (the public half is on the VPS)
ANDROID_KEYSTORE_BASE64	Secret	android.yml	The release keystore, base64 on one line	GitHub; keystore file in password manager
ANDROID_KEYSTORE_PASSWORD	Secret	android.yml	Keystore password	GitHub; password manager
ANDROID_KEY_ALIAS	Secret	android.yml	mobileshop	GitHub
GITHUB_TOKEN	Automatic	deploy.yml images job	Push to ghcr.io (needs packages: write , already set)	Provided by GitHub
APP_URL	Variable	android.yml (required), deploy.yml final check (optional)	https://mobileshop.opsolutions.com	GitHub

The deploy job runs in the GitHub environment production . If you ever add protection rules (required reviewers) to that environment, every release will wait for an approval click.

Variables for the helper scripts

Name	Script	What it does
`API`	`demo-shop.mjs`	API base, default `http://localhost:8000/api/v1`.
`PHONE`, `PASS`, `SHOP`	`demo-shop.mjs`	Login and shop name of the demo shop (defaults: random `0311…` number, `demo12345`, "Demo Mobile Centre").
`ONLY`	`manual-shots.mjs`	Comma list of screenshot names to retake.
`SUPER_PHONE`, `SUPER_PASS`	`manual-admin-shots.mjs`	Super-admin login (default `03990000099`; must be in `SUPERUSER_PHONES`).

4. Daily development commands (Windows)

All commands run on the office PC. The backend needs Python 3.12, the frontend Node 22. Open two terminals: one for the backend, one for the frontend.

> **Important:** Always set `PYTHONUTF8=1` before running any Python on Windows. Without it Urdu text crashes the console (cp1252).

First-time setup of the PC

1. Clone: `git clone https://github.com/AbdurRehmanAfzal/mobile-shop-software.git`
2. Check the repo-local git identity: `git config user.name` must print `Abdur Rehman Afzal` and `git config user.email` must print `abdurrehmanafzal786@gmail.com`.
3. Backend virtual environment (PowerShell, in `backend`): `py -3.12 -m venv .venv`
4. Install the backend with dev tools: `.venv\Scripts\pip install -e .[dev]`
5. Copy the template: `Copy-Item .env.example .env`, then set the local values from chapter 3 (`PUBLIC_SIGNUP=true`, `SCHEDULER_ENABLED=false`, `SUPERUSER_PHONES`).
6. Frontend (in `frontend`): `npm install`
7. Playwright browser for e2e and screenshots (in `frontend`): `npx playwright install chromium`

Run the app locally

What	Where	Command (PowerShell)
UTF-8 for Python (each new terminal)	anywhere	`$env:PYTHONUTF8 = "1"`
UTF-8 in cmd.exe instead	anywhere	`set PYTHONUTF8=1`
Database to the latest version	`backend`	`.venv\Scripts\alembic upgrade head`
Show the local database version	`backend`	`.venv\Scripts\alembic current`
Start the backend on :8000	`backend`	`.venv\Scripts\uvicorn app.main:app --reload --port 8000`
Backend health	browser	`http://localhost:8000/health`
API docs (dev only)	browser	`http://localhost:8000/docs`
First local login without the signup page	`backend`	`.venv\Scripts\python -m app.cli create-owner`
Start the frontend on :5173	`frontend`	`npm run dev`

What	Where	Command (PowerShell)
Frontend for phones on the same Wi-Fi (http)	`frontend`	`npm run dev:lan`, then open `http://<PC-IP>:5173` on the phone
Frontend over https on :5174 (live camera on a phone)	`frontend`	`npm run dev:phone`, then open `https://<PC-IP>:5174` and accept the self-signed certificate
Find the PC's Wi-Fi address	anywhere	`ipconfig`

For `dev:lan` and `dev:phone`, allow Node.js through Windows Firewall the first time it asks. The dev server proxies `/api` and `/health` to `http://localhost:8000`, so the backend must be running. Over plain http (`dev:lan`) the phone's camera falls back to taking a photo; the live scanner needs https (`dev:phone` or the live site).

Tests and checks (run before every commit)

What	Where	Command
Backend lint (the same as CI)	`backend`	`.venv\Scripts\ruff check app tests`
All backend tests	`backend`	`.venv\Scripts\pytest -q`
Backend tests by name	`backend`	`.venv\Scripts\pytest -q -k scenario`
One backend test file	`backend`	`.venv\Scripts\pytest -q tests/test_plan_reminders.py`
Frontend type check	`frontend`	`npm run typecheck`
Frontend unit tests	`frontend`	`npm test`
Production build (catches the 2 MB service-worker precache limit)	`frontend`	`npm run build`
All end-to-end tests (backend on :8000 with `PUBLIC_SIGNUP=true`)	`frontend`	`npm run e2e`
One e2e file	`frontend`	`npx playwright test e2e/crud.spec.ts`
e2e by title	`frontend`	`npx playwright test -g "bill number"`
Report after an e2e failure	`frontend`	`npx playwright show-report`

The e2e suite (`frontend/e2e/*.spec.ts` , Pixel 7 size) starts `npm run dev` itself or reuses a running one. `admin.spec.ts` needs `03990000099` in the local `SUPERUSER_PHONES` .

Migrations on PostgreSQL locally (what CI does)

CI runs every migration up, all the way down and up again on PostgreSQL 16. To reproduce a CI migration failure on the PC, point `DATABASE_URL` at a local PostgreSQL for one terminal (in `backend`):

1. `$env:DATABASE_URL = "postgresql+psycopg://<user>:<password>@localhost:5432/<test_db>"`
2. `.venv\Scripts\pip install -e .[dev,postgres]`
3. `.venv\Scripts\alembic upgrade head`
4. `.venv\Scripts\alembic downgrade base`
5. `.venv\Scripts\alembic upgrade head`
6. `Remove-Item Env:DATABASE_URL`

A new migration

1. Change the models in `backend/app/modules/...` .
2. Generate: `.venv\Scripts\alembic revision --autogenerate -m "short description"`
3. Read the generated file and keep it additive (new tables, new nullable columns, backfills). Never drop or rename a column that production data uses.
4. Apply locally with `.venv\Scripts\alembic upgrade head` , run the tests, and test the downgrade once with `.venv\Scripts\alembic downgrade -1` followed by `upgrade head` .
5. Update the expected head in `docs/29-OWNER-PENDING.md` (section B).

Python scripts on Windows

Heredoc Python (`python - <<EOF`) and heredocs with quotes sometimes hang or fail in this Windows shell. Write the script to a file (for example in your temp folder) and run it with the project's interpreter, with UTF-8 on:

- `$env:PYTHONUTF8 = "1"; backend\.venv\Scripts\python C:\path\to\script.py`

Ports used locally

Port	Who
8000	Backend (uvicorn)
5173	Frontend dev server (`dev` , `dev:lan`)
5174	Frontend over https (`dev:phone`)
8787	One-PC Windows program

Port	Who
8080	CI only: the stack's web container on the runner

5. Git and GitHub

The rules

- **Author:** every commit is authored by `Abdur Rehman Afzal <abdurrehmanafzal786@gmail.com>` only. The repo-local git config is already set to it.
- **No AI attribution, ever:** no `Co-Authored-By` line and no "Generated with ..." line in commits, pull requests or files. This overrides any tool's default.
- **Branch:** work on `develop`. `main` changes only by a pull request from `develop`.
- **Push:** only when the owner asks. Pushing `develop` is safe (CI only); merging into `main` is a production release.
- **Tests first:** run the checks in chapter 4 before committing business logic (ledger, balances, device states, tax).
- **Docs with the code:** update `docs/06-PROGRESS.md`, append to `docs/09-SESSION-LOG.md`, add an ADR to `docs/10-DECISIONS.md` for a design decision, and keep `docs/29-OWNER-PENDING.md` current with every new env key, server command or manual test.
- **Secrets:** `.env` files are git-ignored. Never commit a `.env`, a keystore, a database dump or an uploads folder.

Commit messages

Use a type prefix and say what changed in shop words, as the history does: `feat(lending): ...`, `fix(area): ...`, `docs(manual): ...`, `chore(demo): ...`, `build(frontend): ...`, `seo: ...`.

Everyday git commands

What	Command
Where am I, what changed	`git status`
Be on develop	`git switch develop`
Get the latest develop	`git pull --ff-only origin develop`
See the history	`git log --oneline -15`
Branches, local and remote	`git branch -a`
Remotes	`git remote -v`
Stage specific files	`git add path/to/file1 path/to/file2`
Commit	`git commit -m "feat(area): what changed"`
Push develop (runs CI)	`git push origin develop`
What is on develop but not yet on main	`git log --oneline origin/main..origin/develop`

What	Command
Recent commits that touched a file	`git log --oneline -5 -- path/to/file`
Undo a pushed commit (a new reversing commit)	`git revert <sha>`

Never use `git push --force` on `develop` or `main`, and never rewrite published history.

GitHub CLI on this PC

The GitHub CLI is installed but not in PATH. Call it from PowerShell with its full path:

- `& "$env:ProgramFiles\GitHub CLI\gh.exe" auth status`

In the table below, `gh` stands for `& "$env:ProgramFiles\GitHub CLI\gh.exe"`. For the current PowerShell window you can make a short alias with `Set-Alias gh "$env:ProgramFiles\GitHub CLI\gh.exe"`.

What	Command
Log in (once)	`gh auth login`
Open the release pull request develop → main	`gh pr create --base main --head develop --title "Release: <what ships>" --body "<what ships, what to test>"`
List open pull requests	`gh pr list`
See a pull request	`gh pr view <number>`
Watch a pull request's checks	`gh pr checks <number> --watch`
Merge with a merge commit (starts the release)	`gh pr merge <number> --merge`
Recent runs of every workflow	`gh run list --limit 10`
Recent Deploy runs	`gh run list --workflow Deploy --limit 5`
Recent CI runs on develop	`gh run list --workflow CI --branch develop --limit 5`
Watch a run until it ends, fail if it fails	`gh run watch <run-id> --exit-status`
Summary of a run (jobs, status)	`gh run view <run-id>`
Only the log of the failed steps	`gh run view <run-id> --log-failed`
Re-run only the failed jobs	`gh run rerun <run-id> --failed`
Run Deploy by hand on main	`gh workflow run Deploy --ref main`
Build the Android app by hand	`gh workflow run Android --ref main`
Build the Windows installer with a version	`gh workflow run Windows --ref main -f version=2.1.0`

What	Command
Download all artifacts of a run	`gh run download <run-id>`
Download one artifact by name	`gh run download <run-id> -n MobileShop-Setup`
List secret names (values are never shown)	`gh secret list`
Set or replace a secret (it asks for the value)	`gh secret set VPS_SSH_KEY`
List repository variables	`gh variable list`

What CI checks

On every push to `develop` and every pull request the CI workflow runs three jobs. A red job means the change would not be released.

- **backend:** `pip install -e ".[dev,postgres]"`, `ruff check app tests`, `pytest -q -p no:warnings`, then on a real PostgreSQL 16 service `alembic upgrade head`, `alembic downgrade base`, `alembic upgrade head`.
- **frontend:** `npm ci`, `npm run typecheck`, `npm test`, `npm run build`.
- **stack:** creates a stand-in `n8n_default` network, builds and starts db, backend and web from `deploy/docker-compose.yml` plus `deploy/ci/compose.ci.yml` (web on `localhost:8080`, `PUBLIC_SIGNUP=true`), then runs `deploy/ci/smoke.sh`: health, the app page, a deep link, signup, the seeded catalogue (more than 10 brands), a person, a phone, a report.

Common reasons CI goes red: ruff (an unused import, style), a test that depends on today's date, a migration that does not downgrade, a `package-lock.json` not committed with a new npm package, and the frontend bundle growing past the 2 MB the service worker precaches (the build fails; split the code, as was done for the language files in `a439678`).

6. Releasing to production

A release is a merge of `develop` into `main`. The owner decides when. The push to `main` starts the Deploy workflow, which takes about 10 to 12 minutes from merge to live. A failed check means nothing shipped: production stays on the previous version.

Before you release

1. Everything is committed and pushed on `develop`, and the latest CI run on `develop` is green: `gh run list --workflow CI --branch develop --limit 3`.
2. You know what ships: `git log --oneline origin/main..origin/develop`.
3. If the release has a migration, read it once more: it will run on the production database by itself. It must be additive.
4. If the release adds an environment variable, check that `deploy/docker-compose.yml` passes it into the container (otherwise setting it in `.env` does nothing) and that `docs/29-OWNER-PENDING.md` lists it.
5. Check the server has a fresh backup (chapter 8). For a risky migration, take one by hand first on the server: `/opt/mobileshop/backup.sh`.

Release step by step

1. Open the pull request: `gh pr create --base main --head develop --title "Release: <summary>" --body "<what ships>"`
2. Wait for its checks: `gh pr checks <number> --watch`
3. Merge with a merge commit (not squash, not rebase): `gh pr merge <number> --merge`
4. Find the Deploy run it started: `gh run list --workflow Deploy --limit 1`
5. Watch it: `gh run watch <run-id> --exit-status`
6. Confirm it is live (see "Confirming the release is live" below).
7. Record it in `docs/09-SESSION-LOG.md`: the PR number, the merge commit, the Deploy run id and what you checked on the live site.

What the Deploy workflow does

Job	What happens	If it fails
`checks / backend`, `checks / frontend`, `checks / stack`	The whole CI on the merge commit.	Nothing is built or shipped. Production stays on the previous version.
`android`	Builds and signs the APK (artifact `mobile-shop-android-<date>-<sha>`).	The release still ships, but the new web image has no `/app.apk` (404 until the next good build).

Job	What happens	If it fails
`images`	Copies the APK into `frontend/public/app.apk`, builds and pushes `mobile-shop-backend` and `mobile-shop-web` to ghcr.io with tags `<sha>` and `latest`.	Nothing is deployed.
`deploy`	SSH as `deploy` to the VPS, `sudo -n /opt/mobileshop/deploy.sh <sha>`, then (if `APP_URL` is set) polls `APP_URL/health` for up to 2 minutes.	`deploy.sh` rolls back by itself if the new version is not healthy; see below.

What deploy.sh does on the server

1. Checks the tag looks like a commit sha (or `latest`) and reads the previous tag from `.current-tag`.
2. Downloads `docker-compose.yml`, `backup.sh` and `deploy.sh` of that exact commit from `raw.githubusercontent.com`, so stack changes arrive with the release. If GitHub cannot be reached it keeps the old files and says so.
3. Pulls the backend and web images for the tag, retrying up to 4 times (the registry sometimes drops mid-download).
4. Runs `docker compose up -d --remove-orphans` with `IMAGE_TAG=<sha>`. The backend runs `alembic upgrade head` before it starts serving.
5. Waits up to 90 seconds for `http://localhost/health` inside the web container.
6. Healthy: writes the tag to `.current-tag`, removes dangling images, prints `deploy ok: <sha>`.
7. Not healthy: prints the last 80 backend log lines, starts the previous tag again and fails the job.

> **Warning:** The automatic rollback swaps the images back but does not undo a migration that already ran. Because migrations are additive, the old code normally runs on the newer schema. If it does not, restore the database from the backup taken before the release (chapter 8).

Confirming the release is live

Do not rely on a green run alone; check that the live site serves the new code.

> **Note:** Every `/api/v1/...` path answers 401 without a login, and `/openapi.json` at the root returns the app's `index.html` (the single-page app catches every unknown path), so neither proves which version is live. `/docs` is hidden in production.

1. Health (backend up, production mode): `curl -s https://mobileshop.oopsolutions.com/health` prints `{"status":"ok","env":"prod"}`.

2. Find the current app bundle (Git Bash): `curl -s https://mobileshop.oopsolutions.com/welcome | grep -o '/assets/index-[^"]*\.js'`
3. Look in that bundle for a string the release added, such as a new API path or route (Git Bash): `curl -s https://mobileshop.oopsolutions.com/assets/index-XXXX.js | grep -c 'repairs/labs'` — a count above 0 means the new code is live.
4. The same in PowerShell, step 2: `$h = (Invoke-WebRequest -UseBasicParsing https://mobileshop.oopsolutions.com/welcome).Content; $js = [regex]::Match($h, '/assets/index-[^"]+\.js').Value; $js`
5. The same in PowerShell, step 3: `(Invoke-WebRequest -UseBasicParsing "https://mobileshop.oopsolutions.com$js").Content -match 'repairs/labs'`
6. The APK is there: `curl -sI https://mobileshop.oopsolutions.com/app.apk` answers 200 with `Content-Type: application/vnd.android.package-archive` (about 11 MB).
7. On the server, the live tag is the merge commit: `cat /opt/mobileshop/.current-tag`.

The texts of each language are in their own chunk (to keep the main bundle under 2 MB), so a new translation string may not be in `index-*.js`. Pick a code string such as an API path or a route name.

Phones that already have the site or the PWA open may show the old version until the app is closed fully and opened again (the service worker updates on the next start). The Android app changes only when the new APK is installed.

Releasing by hand

- Re-run a failed Deploy after a flaky step: `gh run rerun <run-id> --failed`
- Deploy the current `main` again without a new commit: `gh workflow run Deploy --ref main`
- Put an older version back (rollback), on the server as root: `/opt/mobileshop/deploy.sh <older-sha>` — read the migration warning above first.
- If the auto-mode of a coding assistant refuses to push `main`, the owner runs the push or the merge himself; `develop` can still be pushed.

7. The server

Getting in

- From the office PC (PowerShell or Git Bash): `ssh <SSH_USER>@<VPS_IP>`
- Then: `cd /opt/mobileshop`
- If SSH does not answer at all, use the browser console in the Hostinger panel (VPS → Terminal).

The `deploy` user is for GitHub only. Do maintenance as root (or a sudo user), never by widening the `deploy` user's sudo rule.

> **Important:** Inside `/opt/mobileshop`, always pass `--env-file .env` and, for any command that starts or recreates containers (`up`, `restart` after an image change), also `IMAGE_TAG=$(cat .current-tag)`. Without `IMAGE_TAG` compose falls back to the `latest` tag, which may be a different version from the one that is live (for example right after an automatic rollback).

First-time setup (already done for production; for a new server)

The `deploy` user and Traefik come from the company website's setup (`oop-solutions/website/deploy/setup-vps.sh`), so that runs first.

1. DNS: in the Hostinger panel, an A record for the app domain → `<VPS_IP>`.
2. SSH key for CI (skip if the website's key is reused), on your PC: `ssh-keygen -t ed25519 -f ~/.ssh/mobileshop_deploy -C deploy@github`
3. Put the public half into `/home/deploy/.ssh/authorized_keys` on the VPS.
4. On the VPS as root, get the repo: `git clone --depth 1 https://github.com/AbdurRehmanAfzal/mobile-shop-software.git /tmp/ms`
5. Run the setup: `bash /tmp/ms/deploy/setup-vps.sh` — it creates `/opt/mobileshop`, copies the compose file and scripts, creates `.env` with a fresh `POSTGRES_PASSWORD` and `SECRET_KEY`, writes the sudoers rule for `deploy` and installs the 02:30 backup cron line.
6. Edit the secrets file: `nano /opt/mobileshop/.env` — set `APP_DOMAIN` and `SUPERUSER_PHONES` (and the other keys from chapter 3 as they become available).
7. Only if the ghcr.io images are private: `docker login ghcr.io -u AbdurRehmanAfzal` (paste a token with `read:packages` when asked).
8. GitHub: secrets `VPS_HOST`, `VPS_USER` (`deploy`), `VPS_SSH_KEY`; variable `APP_URL` (`https://<domain>`).
9. First release: merge `develop` into `main`, or `gh workflow run Deploy --ref main`.
10. First login (signup is closed, so make it on the server; it asks for the password without echoing it): `cd /opt/mobileshop && docker compose --env-file .env exec backend python -m app.cli create-owner` — use the number that is in `SUPERUSER_PHONES`.
11. Production renewal keys, once: `docker compose --env-file .env exec backend python -m app.modules.plan.renewal keygen` — paste both printed lines into `.env` as

`RENEWAL_PRIVATE_KEY` and `RENEWAL_PUBLIC_KEY`, then restart the backend (below).

12. Remove the temporary clone: `rm -rf /tmp/ms`

Day-to-day server commands

All from `/opt/mobileshop` as root.

What	Command
Which version is live	`cat .current-tag`
Containers and their health	`docker compose --env-file .env ps`
Backend log, last 100 lines	`docker compose --env-file .env logs --tail 100 backend`
Follow the backend log live	`docker compose --env-file .env logs -f backend`
Web (Caddy) log	`docker compose --env-file .env logs --tail 100 web`
Database log	`docker compose --env-file .env logs --tail 100 db`
Health from inside the stack	`docker compose --env-file .env exec -T web wget -qO- http://localhost/health`
Health from outside	`curl -s https://mobileshop.oopsolutions.com/health`
Restart the backend (same version, e.g. after a code hang)	`docker compose --env-file .env restart backend`
Apply a changed `.env` to the backend	`IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d backend`
Apply a changed `.env` to the whole stack	`IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d`
Stop the app (keeps data)	`docker compose --env-file .env stop backend web`
Start it again	`IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d`
Deploy a given version by hand	`/opt/mobileshop/deploy.sh <sha>`
Resource use of the containers	`docker stats --no-stream`
Disk space	`df -h /`
Docker's disk use	`docker system df`
Images on the server	`docker image ls 'ghcr.io/abdurrehmanafzal/*'`
Remove unused images (old releases)	`docker image prune -a -f --filter "until=168h"`
Traefik log (certificate, routing)	`docker logs --tail 100 n8n-traefik-1`

What	Command
Backup cron line is present	`crontab -l`
Backup log	`tail -50 /var/log/mobileshop-backup.log`

> **Note:** `restart` keeps the existing container and its old environment. After editing `.env`, use `up -d` (which recreates the container), not `restart`.

Editing the environment file

1. `cp .env .env.bak-$(date +%Y%m%d)` (a dated copy before you change anything; it stays root-only in the same folder)
2. `nano .env`
3. `IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d backend`
4. `docker compose --env-file .env logs --tail 50 backend` — it should end with uvicorn "Application startup complete".
5. Tick the key in `docs/29-OWNER-PENDING.md`.

Keep values on one line, without spaces around `=`. Put a value that contains spaces in double quotes (`backup.sh` reads the same file as a shell script). JSON values such as `WHATSAPP_TEMPLATES` go on one line.

Database shell and queries

What	Command
Open psql	`docker compose --env-file .env exec db psql -U mobileshop -d mobileshop`
Run one query	`docker compose --env-file .env exec -T db psql -U mobileshop -d mobileshop -c "select count(*) from shops;"`
Database size	`docker compose --env-file .env exec -T db psql -U mobileshop -d mobileshop -c "select pg_size_pretty(pg_database_size('mobileshop'));"`
List tables (inside psql)	`\dt`
Leave psql	`\q`

> **Warning:** Never update or delete rows by hand in production. The ledger and device_events are append-only; corrections are new reversing entries made through the app. Use psql to read, and only read, unless a written plan says otherwise and a fresh backup exists.

Migrations on the server

Migrations run automatically every time the backend container starts (`alembic upgrade head` in the image's start command). You never need to run them by hand.

What	Command
Which revision the database is at	`docker compose --env-file .env exec backend alembic current`
Migration history	`docker compose --env-file .env exec backend alembic history --verbose`
Run them by hand (only if a start was interrupted)	`docker compose --env-file .env exec backend alembic upgrade head`

Expected today: `a5c7e9b1d3f6 (head)` . If the backend keeps restarting after a release, the log usually shows the failing migration first (chapter 12).

Server-side app commands

What	Command
Create a shop with its owner login (first login on a new server)	`docker compose --env-file .env exec backend python -m app.cli create-owner`
Make a renewal / license key pair (once)	`docker compose --env-file .env exec backend python -m app.modules.plan.renewal keygen`
Give barcodes to stock entered before barcodes existed (safe to repeat)	`docker compose --env-file .env exec backend python -m app.modules.barcodes.backfill`
A Python shell inside the backend	`docker compose --env-file .env exec backend python`

Files the server keeps

Path	What
`/opt/mobileshop/.env`	All production secrets (mode 600)
`/opt/mobileshop/.current-tag`	The live commit sha
`/opt/mobileshop/docker-compose.yml` , `deploy.sh` , `backup.sh`	Refreshed from GitHub by every deploy
`/opt/mobileshop-backups/`	Nightly dumps and upload archives (14 days)
`/var/log/mobileshop-backup.log`	Backup log
`/etc/sudoers.d/mobileshop-deploy`	The `deploy` user's single allowed command

Path	What
`/home/deploy/.ssh/authorized_keys`	The public half of `VPS_SSH_KEY`
Docker volumes `mobileshop_pgdata`, `mobileshop_uploads`	The database files and the uploads

8. Backups and restore

What runs every night

At 02:30 (server time) cron runs `/opt/mobileshop/backup.sh`, which:

1. Dumps the database with `pg_dump --format=custom` to `/opt/mobileshop-backups/db-YYYYMMDD-HHMM.dump`.
2. Archives the uploads volume to `/opt/mobileshop-backups/uploads-YYYYMMDD-HHMM.tgz`.
3. Deletes files in that folder older than 14 days.
4. If `RCLONE_REMOTE` is set and rclone is installed, copies the last two days' files to that remote and deletes copies older than 30 days there.

The log line `backup ok <stamp>` (and `offsite copy ok ...` with rclone) goes to `/var/log/mobileshop-backup.log`.

Checking backups

On the server:

- Newest files: `ls -lht /opt/mobileshop-backups | head -6`
- Last log lines: `tail -20 /var/log/mobileshop-backup.log`
- Take a backup now: `/opt/mobileshop/backup.sh`
- Off-server copies (when rclone is set up): `rclone ls gdrive-safe: | tail`
- A dump is readable (lists its contents): `docker compose --env-file .env exec -T db pg_restore -l < /opt/mobileshop-backups/db-YYYYMMDD-HHMM.dump | head`

A dump that is only a few bytes, or a log without `backup ok` for last night, means the backup failed: run it by hand and read the error.

Copying a backup to the office PC

- `scp <SSH_USER>@<VPS_IP>:/opt/mobileshop-backups/db-YYYYMMDD-HHMM.dump .`
- `scp <SSH_USER>@<VPS_IP>:/opt/mobileshop-backups/uploads-YYYYMMDD-HHMM.tgz .`

These files contain every shop's data and CNIC photos. Keep them encrypted (a BitLocker drive or an encrypted archive), never in the repository, never in a shared folder.

Setting up the off-server copy (Google Drive, encrypted)

Done once, as root on the VPS. Until this is done, all backups live on the same server as the database.

1. Install rclone: `curl https://rclone.org/install.sh | bash`
2. `rclone config` → `n` (new remote) → name `gdrive` → storage `drive` → leave client id and secret empty → scope `1` (full) → "Use web browser to automatically authenticate?" **n** → it prints an

`rclone authorize "drive" "..."` command: run that on your PC (install rclone there too), log in to the Google account in the browser, paste the token it prints back into the VPS prompt.

3. Encrypt it (the backup holds CNIC photos): `rclone config` → `n` → name `gdrive-safe` → storage `crypt` → remote `gdrive:mobileshop-backups` → filename encryption `standard` → two strong passwords. Store both in the password manager: without them the backups cannot be opened.
4. Switch it on: `echo 'RCLONE_REMOTE=gdrive-safe:' >> /opt/mobileshop/.env`
5. Test: `/opt/mobileshop/backup.sh` — the last line says `offsite copy ok`.
6. Look: `rclone ls gdrive-safe: | tail`

Restore the database (production)

Restoring replaces today's data with the backup's. Every sale made since that backup is lost. Do it only on the owner's decision.

1. Take a backup of the current state first, even if it is broken: `/opt/mobileshop/backup.sh`
2. Stop the app so nothing writes during the restore: `docker compose --env-file .env stop backend web`
3. Restore: `docker compose --env-file .env exec -T db pg_restore -U mobileshop -d mobileshop --clean --if-exists --no-owner < /opt/mobileshop-backups/db-YYYYMMDD-HHMM.dump`
4. Start again with the live version: `IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d`
5. If the backup is from an older release, the backend runs the newer migrations on start; watch `docker compose --env-file .env logs -f backend`.
6. Check `/health`, log in, open a shop and look at today's sales and a party balance.

Restore the uploads

- `docker run --rm -v mobileshop_uploads:/u -v /opt/mobileshop-backups:/b alpine sh -c "cd /u && tar xzf /b/uploads-YYYYMMDD-HHMM.tgz"`

This adds back the files from the archive; files uploaded after it stay.

Restore from Google Drive

1. `mkdir -p /root/restore && rclone copy gdrive-safe: /root/restore --include "db-YYYYMMDD-*"`
2. Then restore from `/root/restore/db-YYYYMMDD-HHMM.dump` as above.

Practise a restore (once a month)

Restore into a separate scratch database, so production is never touched:

1. `docker compose --env-file .env exec -T db createdb -U mobileshop restore_test`
2. `docker compose --env-file .env exec -T db pg_restore -U mobileshop -d restore_test --no-owner < /opt/mobileshop-backups/db-YYYYMMDD-HHMM.dump`

3. `docker compose --env-file .env exec -T db psql -U mobileshop -d restore_test -c "select count(*) from shops;"`
4. `docker compose --env-file .env exec -T db dropdb -U mobileshop restore_test`

Hostinger snapshots

The Hostinger panel can snapshot the whole VPS. Take one before a big server change (an OS upgrade, Docker upgrade, Traefik change). It complements the nightly backup; it does not replace it, because it lives with the same provider.

The one-PC installs

Their data lives on the shop's PC (`%LOCALAPPDATA%\Mobile Shop\MobileShop\mobileshop.db`). The tray menu's "Backup now" writes `backups\mobileshop-<date>.db` plus a Desktop copy. Restore: quit the program, copy a backup over `mobileshop.db` , start again. We have no copy of their data; tell each such shop to keep a USB copy.

9. Android app and the Windows PC version

Android: how it is built

- Every release (Deploy → `android` job, which runs `.github/workflows/android.yml`): `npm ci`, `npm run build:native` (Vite in `native` mode with `VITE_API_URL=vars.APP_URL`, no service worker, then `cap sync android`), then Gradle `assembleRelease` signed with the release key.
- The signed APK becomes `frontend/public/app.apk` inside the web image, so it is served at `https://mobileshop.oopsolutions.com/app.apk`. The Super admin page has Download, WhatsApp and Share buttons for that link.
- It is also kept as the run's artifact `mobile-shop-android-<YYYY.MM.DD>-<sha>` for 90 days.
- Without the keystore secrets the job builds a **debug-signed test APK** (`...-test.apk`) and shows a warning. A phone that installed a test APK cannot be updated by a release-signed one (uninstall first), so production must always have the secrets.

What	How
Build the APK without a release	`gh workflow run Android --ref main`
Download the APK of a run	`gh run download <run-id> -n mobile-shop-android-<YYYY.MM.DD>-<sha>`
Which APK is live	`curl -sI https://mobileshop.oopsolutions.com/app.apk` (size, date)
Check the release key secrets exist	`gh secret list` (look for `ANDROID_KEYSTORE_BASE64`, `ANDROID_KEYSTORE_PASSWORD`, `ANDROID_KEY_ALIAS`)

Android: the release key (one time, keep it forever)

Every update must be signed with the same key, or phones refuse to install it over the old app. Make it once (the VPS has Docker, so Java comes from a container):

1. `mkdir -p /root/android-key`
2. `docker run --rm -it -v /root/android-key:/k eclipse-temurin:17-jdk keytool -genkeypair -v -keystore /k/mobileshop-release.jks -alias mobileshop -keyalg RSA -keysize 2048 -validity 10000 -dname "CN=OOP Solutions, O=OOP Solutions, C=PK"`
3. `base64 -w0 /root/android-key/mobileshop-release.jks; echo`
4. GitHub secrets: `ANDROID_KEYSTORE_BASE64` = that line, `ANDROID_KEYSTORE_PASSWORD` = the password you typed, `ANDROID_KEY_ALIAS` = `mobileshop`.
5. Copy `mobileshop-release.jks` and the password into the password manager, then delete the folder from the server: `rm -rf /root/android-key`

> **Important:** Losing the keystore or its password means no installed app can ever be updated again; every shop would have to uninstall and reinstall.

Android: build on a PC with Android Studio (optional)

Not needed day to day (this PC has no Java or Android SDK; GitHub builds it). For a Play Store bundle:

1. Install Android Studio (it brings JDK 17 and the SDK).
2. In `frontend` (cmd.exe): `set VITE_API_URL=https://mobileshop.oopsolutions.com`
3. `npm run build:native`
4. `npx cap open android`
5. In Android Studio: Build → Generate Signed App Bundle / APK, with the same release keystore.
6. Upload the `.aab` to Play Console (internal testing first).

Icons and splash come from `frontend/assets/`; regenerate with `npx @capacitor/assets generate --android`.

Android: install on a phone

1. Open `https://mobileshop.oopsolutions.com/app.apk` on the phone (or send the file by WhatsApp or USB).
2. Open the downloaded `mobile-shop.apk` and allow "Install unknown apps" for that browser once.
3. For an update, install the new APK over the old one; data is on the server, nothing is lost.

The Windows one-PC program

What	How
Build in GitHub with a version	`gh workflow run Windows --ref main -f version=2.1.0`
It also builds by itself	on a push to `main` that touches `local/`, `backend/` or `frontend/` (version 2.0.0 unless run by hand)
Download the installer	`gh run download <run-id> -n MobileShop-Setup` (inside: `MobileShop-Setup-<version>.exe`)
Build on the PC (PowerShell, repo root)	`.\local\build.ps1 -Version 2.1.0`
Output on the PC	`dist\MobileShop\MobileShop.exe`, and `dist\MobileShop-Setup-<version>.exe` when Inno Setup 6 (`iscc`) is on PATH

The PC build needs Python 3.12, Node 22 and Inno Setup 6. It uses its own virtual environment `backend\.venv-local` with the `local` extra (PyInstaller, pystray, pillow, tzdata). The workflow smoke-tests the bundle on port 8790 (`/health`, the app page, `/api/v1/public/info`) before it uploads it.

For each install you hand out, put the production `RENEWAL_PUBLIC_KEY` in its `settings.env` (so renewal codes from the server work) and, once decided, `SAAS_URL=https://mobileshop.oopsolutions.com` (so it reports to Business).

10. Demo data, landing screenshots and the manuals

All of these scripts live in `frontend/scripts/`, run from the `frontend` folder with Node 22, and talk to the **local** app. Start the backend on :8000 (with `PUBLIC_SIGNUP=true` in `backend/.env`, because every script signs up a fresh shop) and, for screenshots, the frontend on :5173. Playwright's Chromium must be installed (`npx playwright install chromium`).

Demo shop with sample data

`demo-shop.mjs` makes a shop with about two months of data in every module, through the real API (so the ledger, device events, stock batches and price rules all apply): purchases (cash and credit), sales of every kind, returns both ways, a void, ID checks, a tablet and a watch, accessories by the dozen, a loan to another shop, repairs, a bank deposit, supplier payments, expenses and staff (manager and salesman logins).

What	Command (in `frontend`)
A demo shop on the local backend (random owner number)	`node scripts/demo-shop.mjs`
With a chosen login and name (Git Bash)	`PHONE=03001112233 PASS=demo12345 SHOP="Demo Mobile Centre" node scripts/demo-shop.mjs`
The same in PowerShell	`$env:PHONE="03001112233"; $env:PASS="demo12345"; node scripts/demo-shop.mjs`

Each step prints a tick or a cross; a cross is logged and the rest continues. Run it on a fresh local database when you want clean screenshots: stop the backend, rename `backend\mobile_shop_v2.db`, run `.venv\Scripts\alembic upgrade head`, start the backend again.

> **Warning:** The script creates its shop with `/auth/signup`. Production has `PUBLIC_SIGNUP=false`, so pointing `API=https://mobileshop.oopsolutions.com/api/v1` at it is refused (403). A demo shop on production is a deliberate owner decision and would need a different route (a shop opened by the super admin); never turn public signup on in production for it.

Landing page screenshots and video

`landing-assets.mjs` signs up "Al-Madina Mobile Zone", fills it, and takes the landing page's phone screenshots and the sample video.

1. Backend on :8000 and frontend on :5173 running.

2. `node scripts/landing-assets.mjs`
3. Output: `frontend/public/landing/*.png`, `frontend/public/landing/demo.webm`, and `demo.mp4` when Playwright's ffmpeg has an H.264 encoder (otherwise webm only, which Safari does not play).
4. Look at the images, commit them with the landing change, release.

The landing files are excluded from the service-worker precache (they are for visitors, about 13 MB), so their size does not hit the 2 MB limit, but keep them reasonable for visitors on mobile data.

User manual (English, Urdu, Roman Urdu)

Source and layout are described in `docs/manual/README.md`; the screenshot names in `docs/manual/SHOTS.md`.

1. Edit the text in `docs/manual/en.md`, `ur.md`, `roman.md` (the three-part copies in `docs/manual/parts/` are caught by the root `.gitignore` rule `parts/`, so they exist only on the PC that made them; the joined `<lang>.md` files are the ones in git).
2. Retake all screenshots in all three languages (it makes its own demo shop each run): `node scripts/manual-shots.mjs`
3. Retake only some, in some languages (Git Bash): `ONLY=home,sale-done node scripts/manual-shots.mjs en,ur,roman`
4. Build every manual that exists as HTML and A4 PDF: `node scripts/manual-pdf.mjs`
5. Build only some: `node scripts/manual-pdf.mjs ur,admin-en`
6. Output: `docs/manual/Mobile-Shop-User-Manual-<lang>.pdf` and `.html`.

The Urdu PDF uses Noto Nastaliq Urdu from Google Fonts, so build with internet on.

Super-admin manual and its screenshots

1. Put the admin screenshot number in the local `SUPERUSER_PHONES` (default `03990000099`) and restart the backend.
2. Take the shots (Git Bash): `SUPER_PHONE=03990000099 SUPER_PASS=<local-test-password> node scripts/manual-admin-shots.mjs en,roman`
3. Output: `docs/manual/img/<lang>/admin-NN.jpg`, `business-<tab>-N.jpg`, `admin-shop.jpg`, `admin-billing.jpg`, `admin-more.jpg`.
4. Build: `node scripts/manual-pdf.mjs admin-en,admin-roman`

The other books (developer manual, this runbook)

`manual-pdf.mjs` knows the books `en`, `ur`, `roman`, `admin-en`, `admin-roman`, `developer-en` and `operations-en`. It builds each one whose markdown file exists in `docs/manual/`, as `Mobile-Shop-<Book>-Manual-<lang>.pdf`.

- This runbook only: `node scripts/manual-pdf.mjs operations-en`

The markdown subset the script understands: `#` title, `##` and `###` headings, paragraphs, single-level `-` and `1.` lists, tables with a header row, one-line `>` notes, `**bold**`, inline code between single backticks, and `![Caption](shot:name)` for a screenshot. No HTML, no links, no nested lists, no code blocks.

> **Important:** This runbook and its PDF are internal. Do not hand them to shops or put them on the website.

11. Monitoring and health checks

There is no external uptime monitor yet. Until one is set up, these are the checks.

Every day (two minutes)

1. `curl -s https://mobileshop.oopsolutions.com/health` prints `{"status":"ok","env":"prod"}`.
2. Open `https://mobileshop.oopsolutions.com/welcome` and log in to your own shop: Home loads, today's figures show.
3. More → Messages (super admin and your own shop): no pile of "Failed" messages. Failed rows keep the reason (expired WhatsApp token, template not approved, SMTP refused).
4. Super admin → Fees: new fee proofs and shop requests are answered.

Every week (on the server)

1. Containers healthy: `docker compose --env-file .env ps` (all `Up`, `db` and `backend` `healthy`).
2. Backups: `ls -lht /opt/mobileshop-backups | head -6` shows last night's two files; `tail -5 /var/log/mobileshop-backup.log` ends with `backup ok`.
3. Disk: `df -h /` below 80 per cent; `docker system df` for images.
4. Errors in the backend log: `docker compose --env-file .env logs --since 24h backend | grep -iE "error|traceback" | tail -20`
5. Memory: `docker stats --no-stream` (the VPS also runs the website, n8n and the voice agent).

Every month

1. Practise a restore (chapter 8).
2. Remove old release images: `docker image prune -a -f --filter "until=168h"`
3. Check the GitHub Actions minutes and the ghcr.io storage in GitHub → Settings → Billing.
4. Read `docs/29-OWNER-PENDING.md` and tick what is done.

Signals inside the app

- **Audit log** (More → Settings → Audit): `fee_payment/report|verify|reject`, `organisation/lock|unlock|renewal_code`, `shop/backup`, `billing/update`, `plan_reminder/snooze`.
- **Security feed** in Shop audit (owner only).
- **Business console** (Super admin → Business): fees received, shops, one-PC installs that reported in, keys issued.
- **Plan reminder state** for a shop (as its owner): `GET /api/v1/plan/reminder?event=login`.

Suggested (not set up yet)

- A free uptime monitor (for example UptimeRobot or Better Stack) on `https://mobileshop.oopsolutions.com/health` every 5 minutes, alerting the owner's email and WhatsApp.
- A weekly alert if `/var/log/mobileshop-backup.log` has no `backup ok` line for the last day.
- GitHub → Settings → Notifications: email on failed workflow runs (on by default for your own pushes; check it is not muted).

12. When something breaks

First rule: find out what changed. Most incidents follow a release, a `.env` edit or a full disk. Look at `cat /opt/mobileshop/.current-tag`, the last Deploy run, and `docker compose --env-file .env logs --tail 100 backend`.

Symptom → check → fix

Symptom	Check	Fix
Site does not open at all (browser timeout)	Hostinger panel: VPS running? `ssh` works? `docker ps` shows `n8n-traefik-1`?	Reboot from the panel if the VPS hangs. If Traefik is down, the website and n8n are down too: start it from its own compose folder (website setup). Then `IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d`.
502 / 504 Bad Gateway from Traefik	`docker compose --env-file .env ps` — is `web` running?	`IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d web`
App page loads, but every action says it cannot reach the server; `/health` fails	`docker compose --env-file .env ps` and `logs --tail 100 backend`	If the backend restarts in a loop, read the first error: migration failure (restore or fix forward), `SECRET_KEY must be set...` (fix `.env`), database connection refused (see next row).
Backend log: cannot connect to the database	`docker compose --env-file .env ps db` and `logs --tail 50 db`	Disk full is the usual cause (`df -h /`). Free space (old images, old backups), then `IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d`.
Certificate error in the browser	`docker logs --tail 100 n8n-traefik-1` (look for ACME errors); DNS still points to the VPS?	Fix DNS in Hostinger; Traefik retries by itself. Port 80 must stay open for the TLS challenge.
Deploy run red in `checks / ...`	`gh run view <run-id> --log-failed`	Nothing shipped. Fix on `develop`, push, new PR. If it is a flake (network, registry), `gh run rerun <run-id> --failed`.
Deploy red in `images`	Log: registry login or build error	Re-run failed jobs. `packages: write` must stay in deploy.yml.

Symptom	Check	Fix
Deploy red in `deploy`: "VPS_SSH_KEY is not a valid private key" or "Permission denied (publickey)"	`gh secret list`; on the VPS `/home/deploy/.ssh/authorized_keys`	Put the whole private key (with BEGIN/END lines) in `VPS_SSH_KEY` again; make sure its public half is in `authorized_keys`.
Deploy red in `deploy`: "sudo: a password is required"	`cat /etc/sudoers.d/mobileshop-deploy`	Re-run `setup-vps.sh` from a fresh clone (it rewrites the rule), or restore the line `deploy ALL=(root) NOPASSWD: /opt/mobileshop/deploy.sh`.
Deploy log: "could not pull images"	`docker login ghcr.io` still valid? Registry status	Re-run the job (the script already retried 4 times). For private images, log in again with a fresh `read:packages` token.
Deploy log: "new version is not healthy ... rolling back"	The printed backend log lines	The old version is serving again. Fix the cause on `develop` and release again. If a migration ran and the old code fails on it, restore the pre-release backup (chapter 8).
Green Deploy, but the site still shows the old version	Bundle check (chapter 6); `cat .current-tag`	If the server has the new tag: the phone's service worker still has the old shell — close the app fully and reopen, or clear site data. If not: the deploy job did not reach the server; re-run it.
A new `.env` setting has no effect	`grep KEY_NAME docker-compose.yml`	The compose file must pass the key into the container. Add it to `deploy/docker-compose.yml` (on `develop`), release, then `IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d backend`. Remember `restart` does not reload `.env`.
After a `.env` change the site went back to an older or newer version	`docker compose --env-file .env images` vs `cat .current-tag`	It was started without `IMAGE_TAG`, so compose used `latest`. Run `IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d`.

Symptom	Check	Fix
WhatsApp messages stay "Failed"	More → Messages, the failure reason; backend log	Token expired (make a new permanent system-user token), template not approved, or the 24-hour window. Update `.env`, then `up -d backend`.
No messages go out at all, none fail	Is the scheduler running? `logs backend` for scheduler lines	`SCHEDULER_ENABLED` is not passed on the server (always on). Check `WHATSAPP_TOKEN` and `WHATSAPP_PHONE_ID` are both set; without them messages wait for the shop to send by hand.
Emails stay queued	`SMTP_*` in `.env`	Gmail needs an app password and port 587 with STARTTLS.
Assistant says "not switched on"	`ANTHROPIC_API_KEY` set?	Set it, `up -d backend`. "Limit reached" means the shop used `AI_DAILY_LIMIT` calls today.
Business → Keys or "Renewal code": "RENEWAL_PRIVATE_KEY is not set"	`.env`	Run the keygen command (chapter 7), paste both lines, `up -d backend`.
A renewal code is refused on a one-PC install	Its `settings.env` `RENEWAL_PUBLIC_KEY`	It must match the server's public key.
`/app.apk` answers 404	The release's `android` job	It failed (often missing `APP_URL` variable or a Gradle error). Fix, then release again or run Android and the Deploy by hand.
Phones refuse to install the new APK ("App not installed", conflicting package)	Was the old one a test (debug) APK, or was the key changed?	Release key secrets must be the original keystore. A phone with a debug build must uninstall it once.
Camera scanning does not work on a phone	Is it the live https site (or `dev:phone`)?	Live camera needs https. The Caddy `Permissions-Policy` allows the camera for this site only.
Upload fails for a large photo	Size	Caddy refuses bodies over 10 MB. Ask for a smaller photo.
A shop says it is locked	Super admin → that shop → paid-until and grace days	Record the payment or extend; a renewal code for an offline shop. Soft-lock never blocks paying, exporting or the backup.

Symptom	Check	Fix
Super admin menu missing for the owner	`SUPERUSER_PHONES` in `.env`	Add the login number, `up -d backend`, log out and in.
A shop owner or staff member forgot the password	Super admin → the password request	Open the request and use "Set password" (it changes the requester's password).
The super admin forgot his own password	Is there a second number in `SUPERUSER_PHONES`?	Keep two super-admin logins so one can reset the other. `create-owner` makes a new shop and login; it does not reset an existing password.
Disk full	`df -h /`, `docker system df`, `du -sh /opt/mobileshop-backups`	`docker image prune -a -f --filter "until=168h"`; old backups are already cut at 14 days; check the other apps on the VPS.
Backup log shows an error after a `.env` change	`tail -30 /var/log/mobileshop-backup.log`	`backup.sh` sources `.env` as a shell file: a value with spaces must be in double quotes. Fix and run `/opt/mobileshop/backup.sh` by hand.
CI fails on the frontend build with a size error	Build log	The app bundle passed the 2 MB service-worker precache limit: split the code (lazy imports), as for the language chunks.
Local: Python crashes on Urdu text	`$env:PYTHONUTF8`	`$env:PYTHONUTF8 = "1"` in that terminal.
Local: the phone cannot open the dev site	Same Wi-Fi? Firewall?	`npm run dev:lan` (or `dev:phone`), allow Node in Windows Firewall, use the PC's address from `ipconfig`.
Local: e2e or screenshot scripts fail at signup	`PUBLIC_SIGNUP`	Set `PUBLIC_SIGNUP=true` in `backend/.env` and restart the backend.
`gh` is not recognised	PATH	Use `& "$env:ProgramFiles\GitHub CLI\gh.exe"`.

Rolling back a bad release

1. Find the previous good sha: `gh run list --workflow Deploy --limit 5` or `git log --oneline origin/main -5`.

2. On the server: `/opt/mobileshop/deploy.sh <previous-sha>`
3. If the bad release had a migration that the old code cannot live with, stop the app and restore the backup taken before the release (chapter 8).
4. On `develop`: `git revert -m 1 <bad-merge-sha>` or fix forward, then release again. The server will follow `main` again on the next release.

Who to tell

- Shops: if the site is down for more than 15 minutes in business hours, a WhatsApp broadcast to shop owners (the app keeps a list of shops in Super admin) saying it is being fixed and when to expect it back.
- Write every incident in `docs/09-SESSION-LOG.md`: what broke, why, what fixed it, what changes so it does not happen again.

13. Still pending on the owner's side

Summary of `docs/29-OWNER-PENDING.md` as of 2026-11-03. That file is the authority; tick items there.

A. Environment variables on the server (`/opt/mobileshop/.env`)

None of these is confirmed done. After editing, `IMAGE_TAG=$(cat .current-tag) docker compose --env-file .env up -d backend` .

Key	What it unlocks
`SUPERUSER_PHONES`	Check your super-admin number is there.
`ADMIN_WHATSAPP`	Fee-proof, shop-request and password-request messages to you.
`RENEWAL_PRIVATE_KEY` , `RENEWAL_PUBLIC_KEY`	Business → Keys and renewal codes (from keygen, below).
`WHATSAPP_TOKEN` , `WHATSAPP_PHONE_ID` , `WHATSAPP_TEMPLATES`	Automatic WhatsApp: reminders, night report, OTP.
`SMTP_HOST` , `SMTP_PORT` , `SMTP_USER` , `SMTP_PASSWORD` , `SMTP_FROM`	Email.
`ANTHROPIC_API_KEY` , `AI_DAILY_LIMIT`	The assistant and photo reading.
`RCLONE_REMOTE`	The off-server backup copy (with the rclone setup in chapter 8).
`SIGNUP_OTP` , `OTP_DEV_ECHO`	Both `false` on production; check `OTP_DEV_ECHO` is never `true` .

B. Commands on the server, still to run

1. Renewal key pair (once): `docker compose --env-file .env exec backend python -m app.modules.plan.renewal keygen` — paste both lines into `.env` , then `up -d backend` .
2. Barcodes for old stock (once, safe to repeat): `docker compose --env-file .env exec backend python -m app.modules.barcodes.backfill`
3. Confirm the database version: `docker compose --env-file .env exec backend alembic current` — expected `a5c7e9b1d3f6 (head)` .

C. Settings in the app only the owner can fill

- Super admin → Fees → Billing settings: bank name, title, account, IBAN, branch, support phone and WhatsApp (until then shopkeepers see no payment details).
- Super admin → Fees → Payment accounts: Easypaisa, JazzCash, SadaPay.

- Grace days (7 now), and each shop's package, fee and paid-until date.
- Business → Shops: mark one-PC customers as Offline.
- Your own shop: logo and business details, NTN, currency format, barcode prefix and type, item and device settings.
- Every shop that printed stickers since 2026-10-31 must reprint them (those stickers did not scan; fixed 2026-11-01).

D. Manual testing

Nothing has been tested by the owner yet. The list in `docs/29` covers subscription reminders, paying the fee, barcodes and stock count, accessories by the dozen, dealer price, tablets and watches with lock checks, the Business console and license keys, Bulk Excel, the business profile on printed bills, the landing page, len-den and its reminders, repair labs and the repair sticker, the on-screen seller signature and the help "?" sheets. Use a local demo shop (chapter 10) and the newest APK from the latest Deploy run. Read the three user-manual PDFs before giving them to shops, especially the Urdu wording.

E. Decisions waiting

Question	Now
Grace days after plan end: 7 or 15?	7
Freight on a purchase: on the supplier's bill or a separate expense?	On the supplier's bill
Lock check also for used mobiles?	Off (tablets and watches always)
Tablet / watch money card "Money on this phone" → "this device"?	Not changed
One-PC installs report to Business with `SAAS_URL`?	Not set

Also not done, from this runbook

- The Android release keystore secrets: confirm with `gh secret list` that `ANDROID_KEYSTORE_BASE64`, `ANDROID_KEYSTORE_PASSWORD` and `ANDROID_KEY_ALIAS` exist and that the keystore is in the password manager.
- An uptime monitor on `/health` (chapter 11).
- A first practice restore (chapter 8).